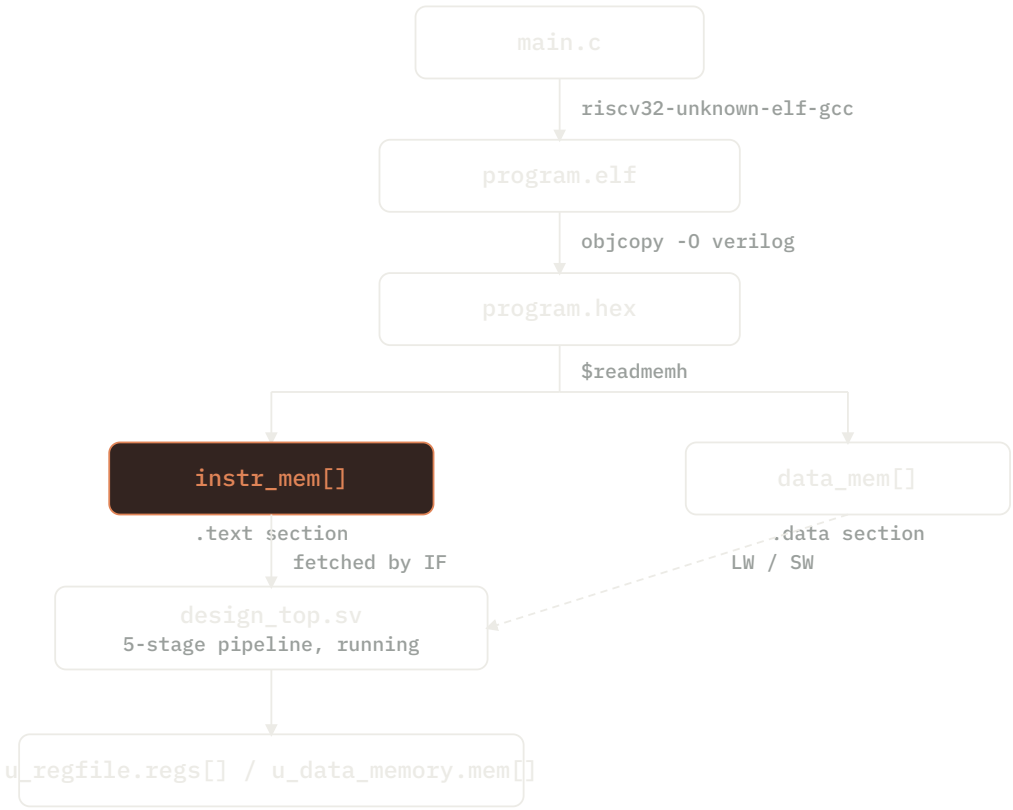
Running compiled C on the RV32I core

Five views of the same example: the toolchain tree that turns a `.c` file into bits in `instr_mem[]`, and the pipeline flow that explains why `JALR` has to exist before a function can return, the full testbench block diagram, how a firmware image would enter the current UVM testbench, and how firmware runs on real hardware with no testbench.

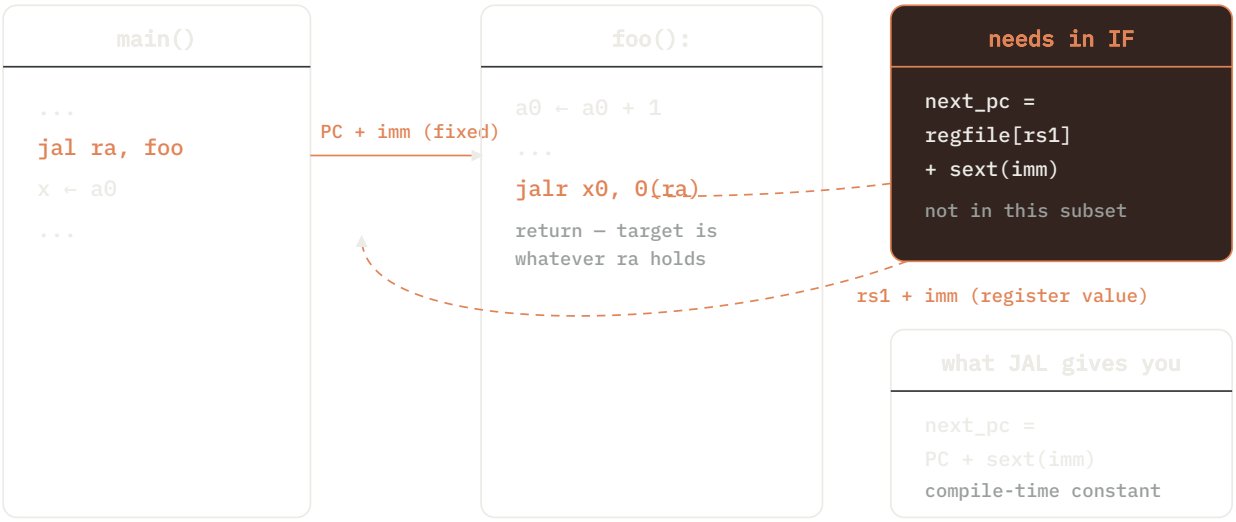
1 — TOOLCHAIN TREE



The same translation tree your directed testbench already walks by hand when it calls `enc_i(...)` — here the branches are generated by the compiler instead of typed into `dut.instr_mem[i]` one line at a time.

---

2 — WHY JALR HAS TO EXIST: CALL/RETURN FLOW

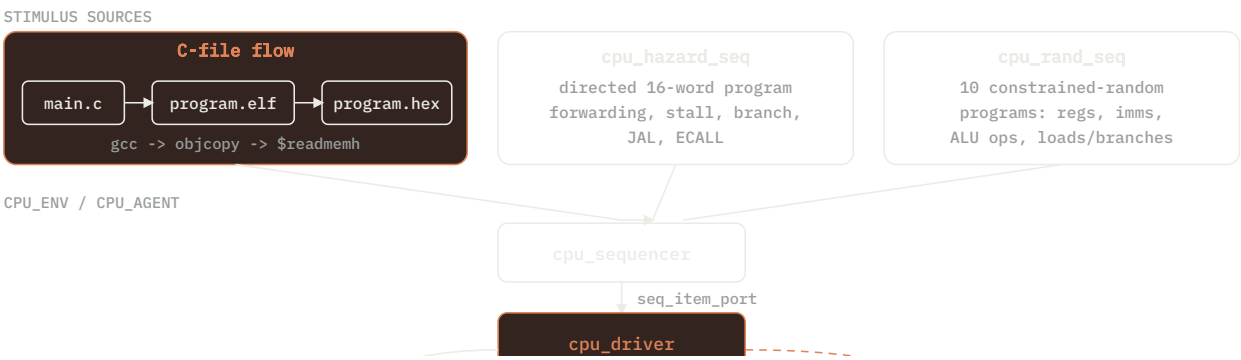


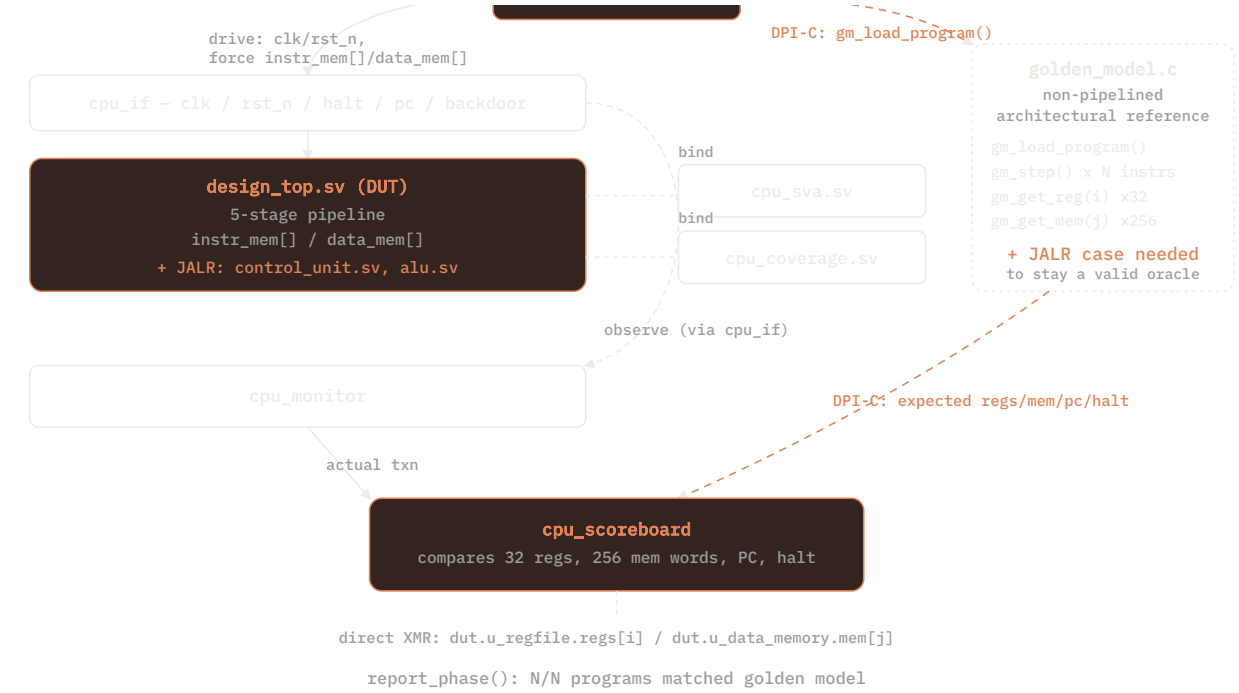
A call-site offset is fixed at compile time, so `JAL` (ALU's `id_ex_pc + 4` link path in `alu.sv`) handles it. A return address is not — it depends on who called — so the return needs a jump whose target comes out of a register (`rs1 + imm`), which is `JALR`. Without it, `foo()` has no instruction that can send control back to whichever call site invoked it.

> Takeaway: the tree is the one-time, off-line translation (compiler → hex → `instr_mem[]`); the flow is what has to happen on-core once that program runs, and it's the flow diagram that exposes the missing instruction — `JAL` alone can issue the call, but only `JALR` can issue the matching return.

---

3 — FULL BLOCK DIAGRAM: C-FILE FLOW INTO THE UVM-TB

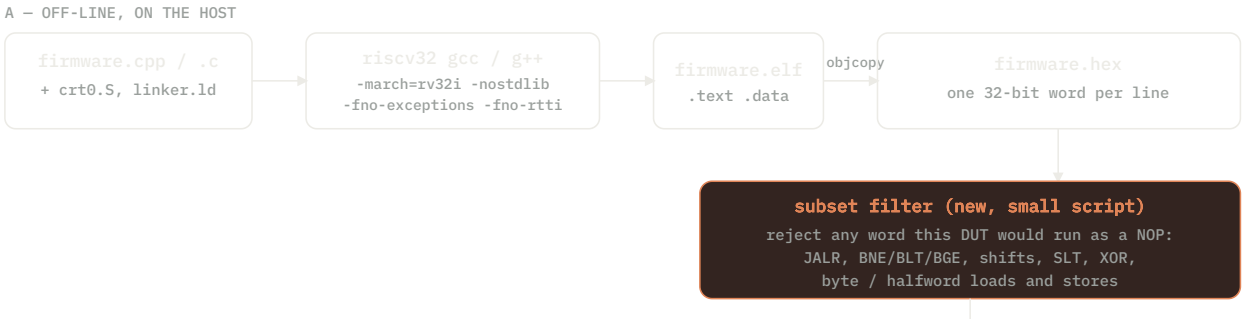


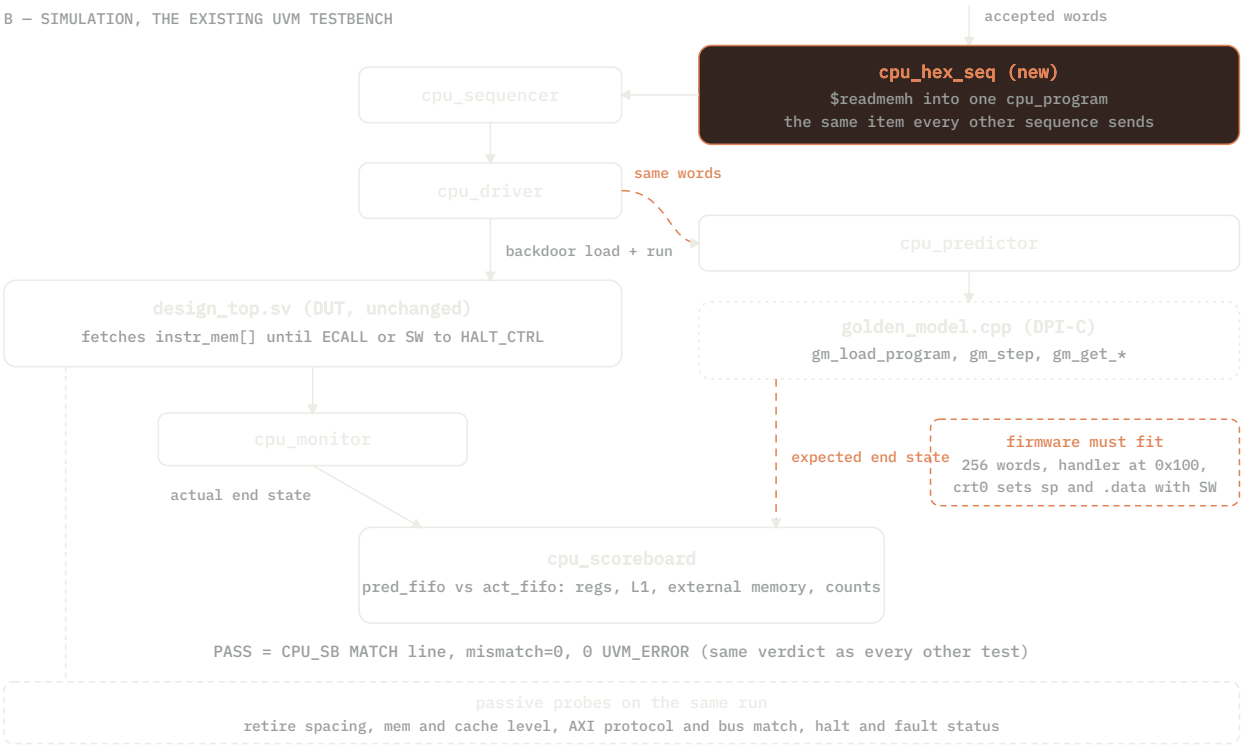


All three stimulus sources converge on the same `cpu_sequencer` → `cpu_driver` path, so a compiled C program is just another sequence item — `cpu_driver` doesn't know or care whether the word array it received came from `cpu_rand_seq` 's constraint solver or from `objcopy` . It forces that array into the DUT's `instr_mem[]` / `data_mem[]` exactly as before, and pushes the same array into `golden_model.c` over DPI-C so both sides execute identical bits. The two points marked in orange are the only real changes a C-call-capable program forces: `JALR` decode in the DUT (`control_unit.sv` , `alu.sv`) and a matching case in `golden_model.c` — without both, `cpu_scoreboard` will flag a mismatch the instant a compiled function tries to return.

---

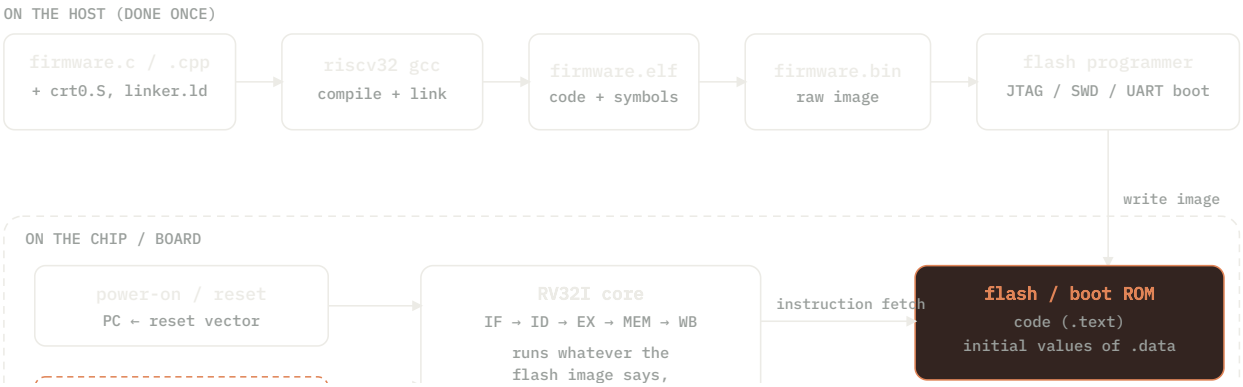
4 — FIRMWARE (C/C++) FLOW INTO THE CURRENT UVM TESTBENCH

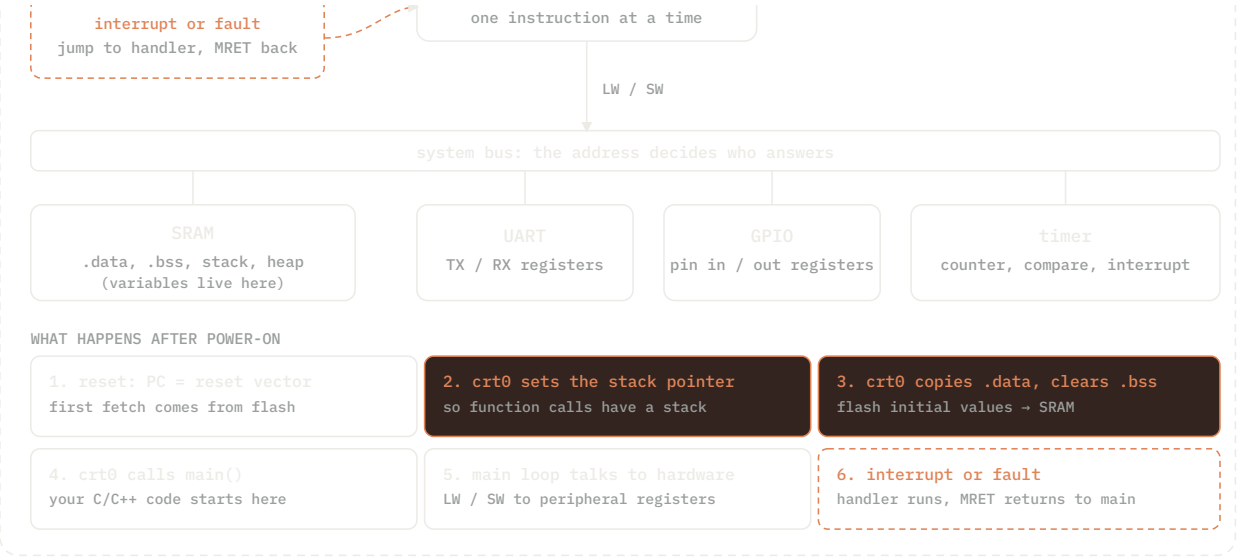




Yes, this is the usual firmware route: C or C++ source and a startup file are cross-compiled and linked to an ELF, the ELF is flattened to a word image, and that image is placed in instruction memory before reset. Here the last step is a testbench sequence. Orange marks the only new pieces (`cpu_hex_seq` and the subset filter) and the limits firmware must respect; everything else already exists. The golden model gets the same words as the DUT, so the scoreboard needs no change. Not built yet: today firmware is limited to what the DUT really executes, because a normal compiler emits `JALR` for every function return and branch forms this core treats as NOPs.

5 — WITHOUT A TESTBENCH: HOW FIRMWARE RUNS ON REAL HARDWARE





This is the same program as diagram 4, minus the testbench. The image is written into flash once. After that, every power-up or reset starts the core at the reset vector, and the startup code (`crt0`) prepares memory so C can run. Peripherals are not special instructions: they are registers at fixed addresses, read and written with ordinary loads and stores, which is also how this DUT's CSR window at `0xFFFFFF00` works. A debugger such as gdb reaches the chip over JTAG or SWD to halt it, read registers and reflash. Compared with diagram 4: the flash replaces the testbench's backdoor load, and nothing checks the result automatically; a person or a test script has to watch the outputs.